

Python/Pygame Marble Tilt Maze Game

Prompts and Responses

Name: Praneetha Sreedharala

SRN: PES1UG24AM281

Section: E

Compiled from the conversation, in chronological order.

Prompt 1

USER PROMPT

I'm working on a Python/Pygame Marble Tilt Maze game. The README says the marble sometimes gives a phantom bounce near wall corners. Can you help me fix the collision detection using accurate circle-vs-rectangle collision detection while keeping the existing game behavior unchanged?

ASSISTANT RESPONSE

Yes. Please upload the current project files (or the .py file containing the marble movement/collision code), especially the README and the main Pygame file.

I'll keep the existing behavior unchanged and only replace the inaccurate collision check with proper circle-vs-rectangle collision detection:

1. Find the closest point on each wall rectangle to the marble's center.
2. Measure the distance from the marble center to that point.
3. Treat it as a collision only when that distance $\leq$ marble radius.
4. Preserve the current bounce/velocity behavior.
5. Pay special attention to wall corners, which is where the phantom bounce is happening.

Once you upload it, I can point out exactly what to replace and give you the corrected code.

Prompt 2

USER PROMPT

I don't need you to modify the files right now. Please explain the recommended implementation for this issue in a concise way, including how circle-vs-rectangle collision detection would prevent the phantom corner bounce.

ASSISTANT RESPONSE

The recommended fix is to replace the current approximate collision check with true circle-vs-rectangle collision detection.

For the marble:

- Center = (cx, cy)

- Radius = r

For each wall rectangle:

1. Find the closest point on the rectangle to the marble's center:

$\text{closest_x} = \max(\text{rect.left}, \min(\text{cx}, \text{rect.right}))$

$\text{closest_y} = \max(\text{rect.top}, \min(\text{cy}, \text{rect.bottom}))$

2. Calculate the distance between the marble center and that closest point:

$\text{dx} = \text{cx} - \text{closest_x}$

$\text{dy} = \text{cy} - \text{closest_y}$

$\text{distance_sq} = \text{dx} * \text{dx} + \text{dy} * \text{dy}$

3. A collision occurs only if:

$\text{distance_sq} \leq r * r$

Why this fixes the phantom bounce:

The problem with simpler collision methods is that they may treat the marble as colliding with the entire rectangular boundary, including areas around corners where the marble isn't actually touching the wall.

Circle-vs-rectangle detection checks the actual shortest distance between the marble and the rectangle.

So near a corner, if the marble is close to the corner but its circular edge doesn't actually touch the rectangle, the distance check returns no collision, so there is no unwanted bounce. If the circle genuinely overlaps the corner, the closest-point calculation detects it and the normal bounce behavior occurs.

Only the collision test should change. Keep the existing marble movement, gravity/tilt acceleration, velocity, bounce/reflection, friction, and wall layout exactly as they are.

Conceptually:

OLD:

```
if approximate_collision(marble, wall):  
    existing_bounce_logic()
```

NEW:

```
if circle_rect_collision(marble_center, marble_radius, wall):  
    existing_bounce_logic()
```

This isolates the fix to collision detection and should eliminate the phantom corner bounce without changing the rest of the game's behavior.

Prompt 3

USER PROMPT

The collision fix makes sense. Now I want to improve the game-over behavior. When the marble reaches the goal, I want a clear graphical "Maze Solved" screen showing the finish time. When the timer expires, it should show a "Time's Up" screen instead of only printing a message in the console. The player should be able to press a key to exit or continue

ASSISTANT RESPONSE

I'd recommend handling this with a simple game-state system rather than printing messages and immediately exiting.

Have three states:

PLAYING

SOLVED

TIME_UP

During the game:

```
if marble_reaches_goal:  
    game_state = "SOLVED"  
    finish_time = elapsed_time  
elif elapsed_time >= time_limit:  
    game_state = "TIME_UP"
```

Then your main drawing loop decides what to display:

```
if game_state == "PLAYING":  
    draw_maze()  
    draw_marble()  
    draw_timer()  
elif game_state == "SOLVED":
```

```
    draw_solved_screen(finish_time)
elif game_state == "TIME_UP":
    draw_time_up_screen()
```

Maze Solved screen:

```
MAZE SOLVED!
Finish Time: 18.42s
Press R to play again
Press ESC to exit
```

Time's Up screen:

```
TIME'S UP!
Better luck next time
Press R to try again
Press ESC to exit
```

Key handling:

```
if game_state in ("SOLVED", "TIME_UP"):
    if event.key == pygame.K_ESCAPE:
        running = False
    elif event.key == pygame.K_r:
        reset_game()
        game_state = "PLAYING"
```

The important part is that you save the finish time when the goal is reached rather than continuing to calculate it afterward.

Once the marble reaches the goal or the timer expires, stop updating the marble and timer. Otherwise the game could continue running underneath the end screen.

This keeps the existing gameplay intact while making the two game-over conditions much clearer and more polished.

Prompt 4

USER PROMPT

The game-over screen is working. Now I want to add a replay option so the player can choose different difficulty levels after a game ends. Please add Easy, Medium, and Hard modes, where each mode changes the tilt strength, friction, and time limit. Starting a new round should reset the marble position, velocity, timer, and game state correctly.

ASSISTANT RESPONSE

Yes. The cleanest approach is to make difficulty a configuration, and have `reset_game()` use that configuration.

1. Define the difficulty settings:

```
DIFFICULTIES = {  
    "Easy": {  
        "tilt_strength": 0.20,  
        "friction": 0.90,  
        "time_limit": 60,  
    },  
    "Medium": {  
        "tilt_strength": 0.35,  
        "friction": 0.85,  
        "time_limit": 45,  
    },  
    "Hard": {  
        "tilt_strength": 0.50,  
        "friction": 0.80,  
        "time_limit": 30,  
    }  
}
```

These numbers are examples—you can keep/tune the values based on your existing physics.

2. Track the selected difficulty:

```
current_difficulty = "Medium"
```

Then whenever a round starts:

```
settings = DIFFICULTIES[current_difficulty]  
tilt_strength = settings["tilt_strength"]  
friction = settings["friction"]  
time_limit = settings["time_limit"]
```

Your existing marble physics should use these variables instead of hard-coded values.

3. Make `reset_game()` reset everything:

```
def reset_game():  
    global marble_x, marble_y  
    global velocity_x, velocity_y  
    global start_time, finish_time  
    global game_state  
    global tilt_strength, friction, time_limit
```

```
marble_x = START_X  
marble_y = START_Y
```

```
velocity_x = 0  
velocity_y = 0
```

```
start_time = pygame.time.get_ticks()  
finish_time = 0
```

```
game_state = "PLAYING"
```

```
settings = DIFFICULTIES[current_difficulty]  
tilt_strength = settings["tilt_strength"]  
friction = settings["friction"]  
time_limit = settings["time_limit"]
```

4. Add a difficulty-selection screen after SOLVED or TIME_UP:
PLAY AGAIN?

1. EASY
2. MEDIUM
3. HARD

ESC - Exit

Key handling:

```
if game_state in ("SOLVED", "TIME_UP"):  
    if event.key == pygame.K_1:  
        current_difficulty = "Easy"  
        reset_game()  
    elif event.key == pygame.K_2:  
        current_difficulty = "Medium"  
        reset_game()  
    elif event.key == pygame.K_3:  
        current_difficulty = "Hard"  
        reset_game()  
    elif event.key == pygame.K_ESCAPE:  
        running = False
```

You can use clickable buttons later, but keyboard selection is the simplest reliable implementation.

5. Show the current difficulty during gameplay:

```

difficulty_text = font.render(
    f"Difficulty: {current_difficulty}",
    True,
    WHITE
)
screen.blit(difficulty_text, (20, 20))

```

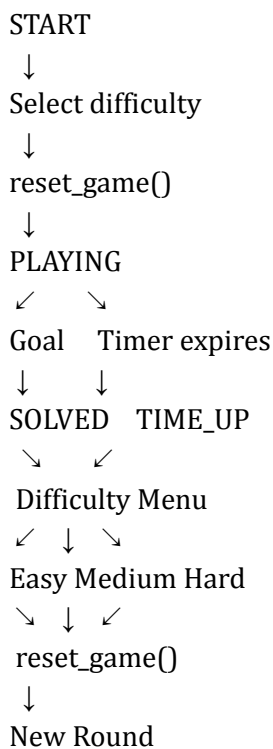
The timer continues using the selected limit:

```

elapsed_time = (pygame.time.get_ticks() - start_time) / 1000
if elapsed_time >= time_limit:
    game_state = "TIME_UP"

```

Overall flow:



The key design point is not to create separate game logic for Easy/Medium/Hard. Keep one game and simply change its physics parameters through DIFFICULTIES.

Prompt 5

USER PROMPT

The replay and difficulty system is done. Now I want to add sound feedback to the game. Please add separate sounds for wall bounces, reaching the goal, and the timer running out.

The sounds should trigger only when those events happen and should not interfere with the existing game logic.

ASSISTANT RESPONSE

Yes. The safest approach is to keep sound completely separate from the physics/game-state logic: detect an event, then play its sound once.

1. Initialize Pygame audio after `pygame.init()`:
`pygame.mixer.init()`

```
bounce_sound = pygame.mixer.Sound("sounds/bounce.wav")
goal_sound = pygame.mixer.Sound("sounds/goal.wav")
timeout_sound = pygame.mixer.Sound("sounds/timeout.wav")
```

Project structure:

```
project/
├── main.py
└── sounds/
    ├── bounce.wav
    ├── goal.wav
    └── timeout.wav
```

You can adjust volume if needed:

```
bounce_sound.set_volume(0.5)
goal_sound.set_volume(0.7)
timeout_sound.set_volume(0.7)
```

2. Play the bounce sound only on an actual collision:

```
if circle_rect_collision(marble, wall):
    bounce_sound.play()
```

```
# Existing bounce/velocity logic
velocity_x = ...
velocity_y = ...
```

Do not put `bounce_sound.play()` inside the main game loop by itself, because then it could play hundreds of times per second.

3. Goal sound:

```
if marble_reaches_goal:
    goal_sound.play()
    finish_time = elapsed_time
    game_state = "SOLVED"
```

Because the state changes from PLAYING to SOLVED, the goal condition won't continuously run afterward.

4. Timeout sound:

```
if elapsed_time >= time_limit:  
    timeout_sound.play()  
    game_state = "TIME_UP"
```

An even safer pattern is:

```
if game_state == "PLAYING":  
    if elapsed_time >= time_limit:  
        timeout_sound.play()  
        game_state = "TIME_UP"
```

5. Don't put sounds inside reset_game(). Keep reset_game() responsible for marble position, velocity, timer, difficulty, and game state, and keep sounds tied to events.

Architecture:

Marble hits wall



Collision detected



bounce_sound.play()



Existing bounce physics

Marble reaches goal



goal_sound.play()



game_state = SOLVED

Timer expires



timeout_sound.play()



game_state = TIME_UP

If the marble can touch multiple walls during a single frame, you may want to ensure the bounce sound doesn't play multiple times in that frame:

bounce_played = False

```

for wall in walls:
    if circle_rect_collision(marble, wall):
        # existing collision logic
        if not bounce_played:
            bounce_sound.play()
            bounce_played = True

```

That gives one bounce sound per frame while leaving the existing collision behavior untouched.

Prompt 6

USER PROMPT

I've now addressed the four main requirements: collision detection, game-over screens, replay with difficulty levels, and sound feedback. Please do a final review of the overall design and point out any obvious bugs, missing requirements, or game-state issues that I should check before submitting the project. Keep the review concise.

ASSISTANT RESPONSE

Based on the changes discussed, the overall design is solid. Before submitting, do this final checklist:

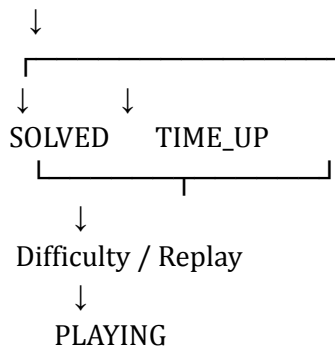
1. Collision detection

- Make sure collision uses circle-vs-rectangle, not just rectangle overlap.
- Check corners specifically — the marble should not bounce when it is merely near a corner.
- Make sure the existing bounce/velocity behavior hasn't changed accidentally.

2. Game states

The flow should be:

PLAYING



Check that:

- Marble stops moving after SOLVED/TIME_UP.
- Timer stops after the game ends.
- Goal and timeout cannot both trigger.
- Restart returns to PLAYING.

3. Difficulty/replay

For Easy / Medium / Hard, verify that each actually changes:

- Tilt strength
- Friction
- Time limit

When restarting, confirm:

Marble position ✓

Velocity ✓

Timer ✓

Game state ✓

Difficulty settings ✓

Also make sure the previous round's finish time doesn't appear in the new round.

4. Game-over screens

Maze Solved:

- Displays correctly.
- Shows the actual finish time.
- Offers replay/difficulty selection and exit.

Time's Up:

- Appears when the timer reaches the limit.
- Doesn't just print to the console.
- Offers replay/difficulty selection and exit.

5. Sound

Check that:

- Bounce sound → only on an actual wall collision.
- Goal sound → once when the goal is reached.
- Timeout sound → once when time expires.
- Sounds don't play repeatedly every frame.
- Missing sound files don't crash the game unexpectedly.

Biggest manual tests:

1. Run into a wall corner → no phantom bounce.
2. Reach goal normally → solved screen + one goal sound.

3. Let timer reach zero → time-up screen + one timeout sound.
4. Replay with a different difficulty → position, velocity and timer completely reset.
5. Press exit from every end screen → game closes cleanly.

If all five pass, the four requested requirements are covered and the game-state design is in good shape.